

南华大学

计算机学院

课程设计报告

(2024 ~2025 学年度 第二学期)

课程名称：操作系统原理

设计题目：进程管理系统设计

专业：物联网工程 班级：本 23 物联 03 班

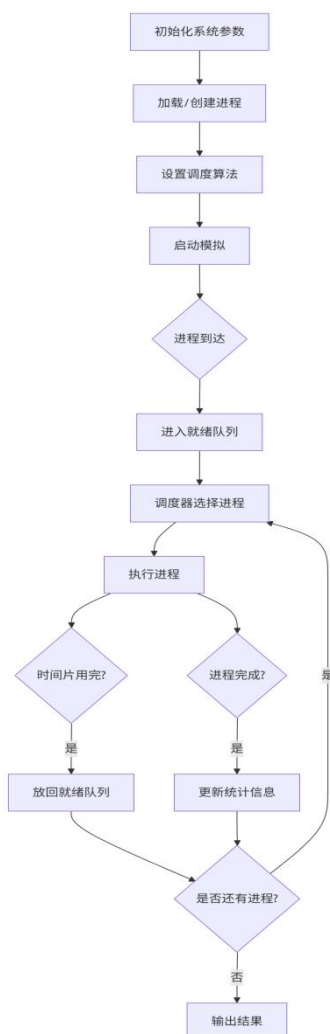
分组成员	负责任务	组内自评	老师评分
铁胜瑄	组长 进程控制 模块设计，实现进程 全生命周期管理	优秀	
王熙	四种调度算法内 容设计，评估相应性 能	优秀	
谭云浩	同步与通信内容 设计，P 和 V 操作和 消息队列实现	优秀	
廖文超	主程序与界面设 计，设计出交互界 面，衔接各个主要算 法模块	优秀	

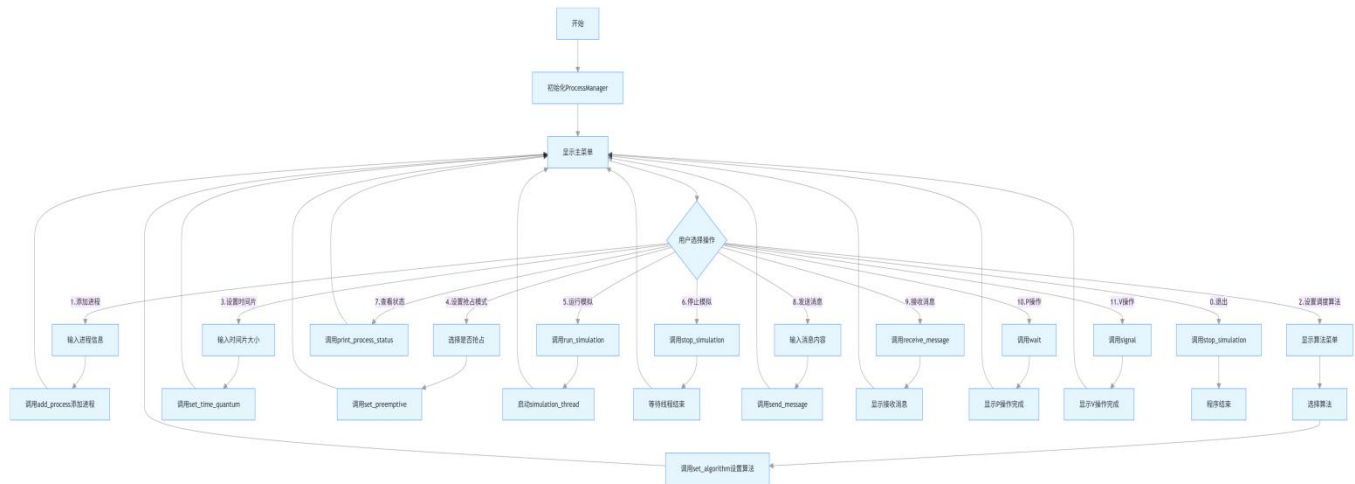
1. 题目要求描述

本设计的目的是加深对进程概念及进程管理各部分内容的理解；熟悉进程管理中主要数据结构的设计及进程调度算法、进程控制机构、同步机构及通讯机构的实施。要求设计一个允许 n 个进程并发运行的进程管理模拟系统。该系统包括有简单的进程控制、同步与通讯机构，其进程调度算法可任意选择。每个进程用一个PCB表示，其内容根据具体情况设置。各进程之间有一定的同步关系（可选）。系统在运行过程中应能显示或打印各进程的状态及有关参数的变化情况，以便观察诸进程的运行过程及系统的管理过程。

2. 程序流程图及源代码注释

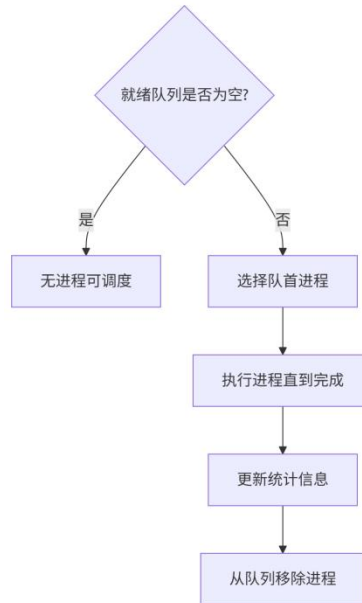
(1) 系统总体流程图





(2) 进程调度算法流程图

①先来先服务 (FCFS) 算法



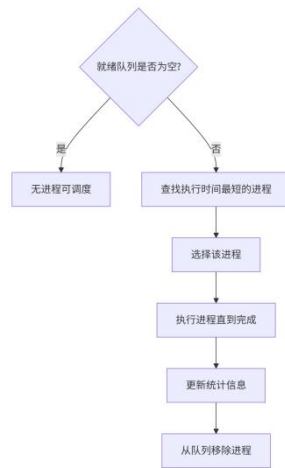
代码:

```

1 Process* ProcessManager::select_next_process() {
2     if (algorithm == SchedulingAlgorithm::FCFS) {
3         if (!ready_queue.empty()) {
4             Process* p = ready_queue.front();
5             ready_queue.pop_front(); // FIFO 规则
6             return p;
7         }
8     }
9 }

```

②短作业优先 (SJF) 算法



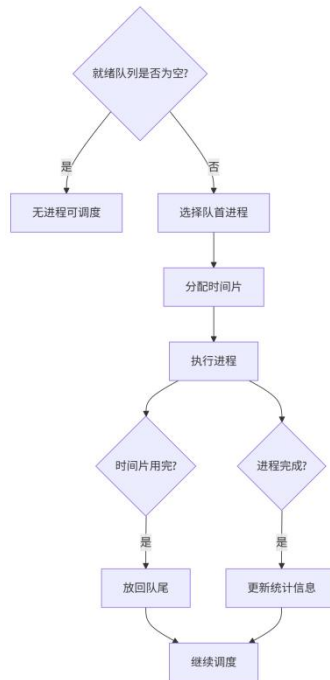
代码:

```

01 Process* ProcessManager::select_sjf_process() {
02     if (!ready_queue.empty()) {
03         // 按剩余时间升序排序，取最小值
04         auto it = std::min_element(ready_queue.begin(),
05 ready_queue.end(),
06             [](Process* a, Process* b) {
07                 return a->remaining_time <
08 b->remaining_time;
09             });
10         Process* selected = *it;
11         ready_queue.erase(it);
12         return selected;
13     }
    return nullptr;
}

```

③时间片轮转(RR)算法



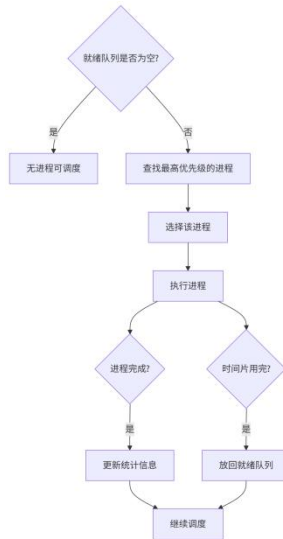
代码:

```

01 void ProcessManager::execute_time_unit() {
02     if (algorithm == SchedulingAlgorithm::RR &&
03         running_process) {
04         running_process->time_quantum++;
05         if (running_process->time_quantum >= time_quantum)
06         {
07             running_process->state = ProcessState::READY;
08             ready_queue.push_back(running_process); // 回
09             到队尾
10             running_process = nullptr;
11             running_process->time_quantum = 0; // 重置计数
12             器
        }
    }
    // 其他逻辑...
}

```

④优先级调度算法

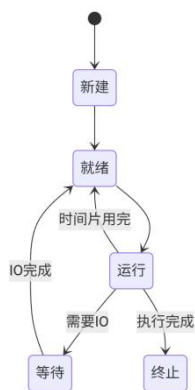


代码:

```

01 Process* ProcessManager::select_priority_process() {
02     if (!ready_queue.empty()) {
03         // 按优先级升序排序, 取最小值
04         auto it = std::min_element(ready_queue.begin(),
05 ready_queue.end(),
06     [](Process* a, Process* b) {
07         return a->priority < b->priority;
08     });
09     Process* selected = *it;
10     ready_queue.erase(it);
11     return selected;
12 }
13 return nullptr;
}
  
```

(3) 进程转换状态图



解释:

(1) 进程状态通过ProcessState枚举定义:

```
enum class ProcessState {
```

```

NEW,          // 新建状态
READY,        // 就绪状态
RUNNING,      // 运行状态
WAITING,       // 等待状态
TERMINATED    // 终止状态
};

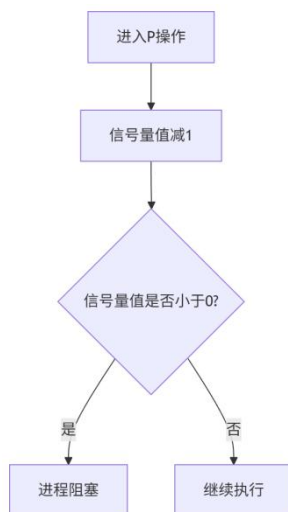
```

(2) 状态转换在以下场景发生:

- 新建 -> 就绪: 进程到达时间等于当前时间
- 就绪 -> 运行: 被调度器选择
- 运行 -> 就绪: 时间片用完 (RR 算法) 或被抢占
- 运行 -> 终止: 进程执行完毕
- 运行 -> 等待: 等待资源 (本系统未完全实现)
- 等待 -> 就绪: 资源可用 (本系统未完全实现)

(4) 信号量操作流程

①P操作(wait)



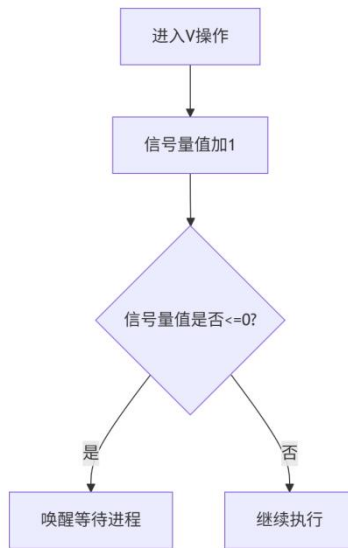
代码:

```

1 void Semaphore::wait() {
2     std::unique_lock<std::mutex> lock(mtx);
3     value--;
4     if (value < 0) {
5         cv.wait(lock); // 阻塞直至被唤醒
6     }
7 }

```

②V操作 (signal)



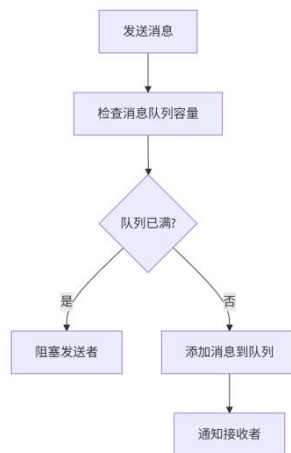
代码:

```

1 void Semaphore::signal() {
2     std::unique_lock<std::mutex> lock(mtx);
3     value++;
4     if (value <= 0) {
5         cv.notify_one(); // 唤醒一个等待线程
6     }
7 }
  
```

(5) 消息通信流程图

①发送消息



代码:

```

1 void ProcessManager::send_message(int msg) {
  
```

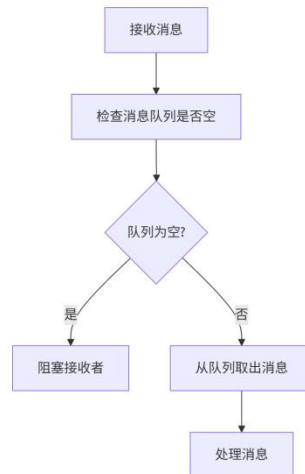


```

2      std::lock_guard<std::mutex> lock(message_mutex);
3      if (running_process) {
4          message_queue.push(Message(running_process->pid,
5 msg));
6          message_semaphore.signal(); // 通知有新消息
7          std::cout << "进程 " << running_process->pid << " 发
8 送消息: " << msg << std::endl;
          }
      }

```

②接收消息



代码:

```

01 int ProcessManager::receive_message() {
02     message_semaphore.wait(); // 阻塞等待消息
03     std::lock_guard<std::mutex> lock(message_mutex);
04     if (!message_queue.empty()) {
05         Message msg = message_queue.front();
06         message_queue.pop();
07         std::cout << "进程 " << running_process->pid << " 收
08 到消息: " << msg.content << std::endl;
09         return msg.content;
10     }
11     return -1;
}

```

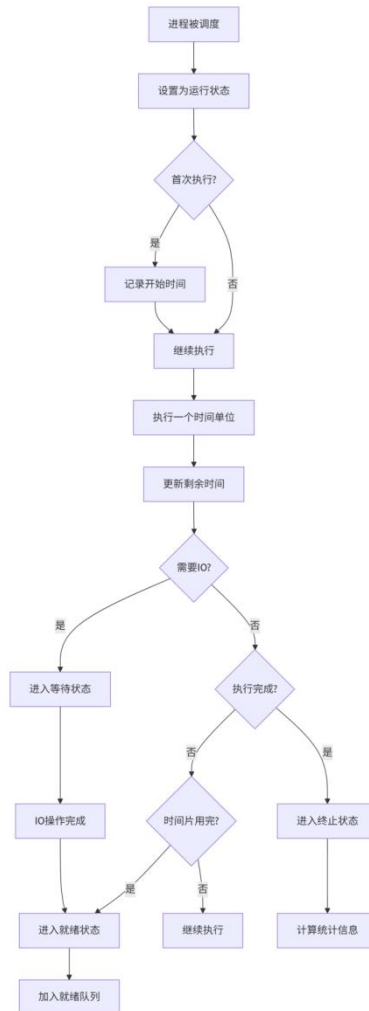
(6) 系统初始化流程图



代码:

```
1 ProcessManager::ProcessManager()  
2     : running_process(nullptr),  
3     algorithm(SchedulingAlgorithm::FCFS),  
4     time_quantum(2),  
5     preemptive(false),  
6     current_time(0),  
7     simulation_running(false),  
8     message_semaphore(0) {  
9 }
```

(7) 进程执行流程图



代码:

```

01 void ProcessManager::execute_time_unit() {
02     if (running_process) {
03         running_process->remaining_time--;
04         if (running_process->remaining_time == 0) {
05             running_process->state =
06 ProcessState::TERMINATED;
07             running_process->finish_time = current_time +
08 1;
09             running_process = nullptr; // 进程执行完毕
10         }
    }
}

```

3. 程序设计

3.1 系统架构设计

本系统采用面向对象设计方法，主要包含以下几个核心组件：

1. Process 类：表示进程控制块（PCB），包含进程的所有状态和参数
 - 基本属性：PID、到达时间、执行时间、优先级
 - 运行状态：新建、就绪、运行、等待、终止
 - 统计信息：开始时间、完成时间、周转时间、等待时间
2. ProcessManager 类：核心管理类，实现进程调度、控制、同步与通信
 - 进程管理：添加、调度、执行进程
 - 调度算法：FCFS、SJF、RR、优先级调度
 - 同步机制：信号量实现进程同步
 - 通信机制：消息队列实现进程间通信
3. 辅助类：
 - Message 类：表示进程间传递的消息
 - Semaphore 类：实现信号量机制
 - SchedulingAlgorithm 枚举：定义调度算法类型
 - ProcessState 枚举：定义进程状态类型

3.2 数据结构设计

系统中使用的主要数据结构：

- `vector<Process>` : 存储所有进程的 PCB
- `deque<Process*>` : 就绪队列，存储可执行的进程
- `queue<Message>`: 消息队列，存储进程间通信的消息
- `mutex` : 互斥锁，保护共享数据结构

`thread` 模拟线程，运行进程调度逻辑

3.3 调度算法实现分析

(1) 先来先服务（FCFS）算法

1) 核心思想：按照进程到达的先后顺序进行调度，先到达的进程先执行

2) 实现方式：

就绪队列采用 FIFO（先进先出）原则

`select_next_process`方法直接选择队列头部进程

3) 特点：实现简单，公平性好，但可能导致短作业等待时间过长

(2) 短作业优先（SJF）算法

1) 核心思想：选择就绪队列中剩余执行时间最短的进程优先执行

2) 实现方式：

使用`std::min_element`算法查找剩余时间最短的进程

`select_sjf_process`方法实现具体选择逻辑

3) 特点：平均等待时间和平均周转时间较短，但可能导致长作业饥饿

(3) 时间片轮转（RR）算法

1) 核心思想：每个进程分配一个时间片，时间片用完后进程重新进入就绪队列

2) 实现方式：

设置`time_quantum`属性表示时间片大小

在execute_time_unit中检查时间片是否用完

时间片用完后将进程重新放入就绪队列

3)特点：响应时间快，适合交互式系统，上下文切换开销较大

(4) 优先级调度算法

1)核心思想：根据进程优先级选择执行，优先级高的进程优先执行

2)实现方式：

进程包含priority属性表示优先级

使用std::min_element算法查找优先级最高的进程

select_priority_process方法实现具体选择逻辑

3)特点：重要进程优先执行，但可能导致低优先级进程饥饿

3.4 进程同步与通信实现

(1) 信号量机制

实现方式：

Semaphore类封装信号量的 P 操作 (wait) 和 V 操作 (signal)

使用互斥锁和条件变量实现信号量的原子操作

应用场景：

进程间同步：确保进程按顺序执行

消息接收等待：阻塞等待消息到达

(2) 消息队列通信

实现方式：

Message类定义消息结构，包含发送者 PID 和消息内容

message_queue队列存储消息，使用互斥锁保护

send_message和receive_message方法实现消息发送和接收

应用场景：

进程间数据传递：模拟进程间通信场景

同步协作：通过消息传递实现进程间协作

3.5核心模块设计

(1) 进程调度模块

1)算法切换：通过set_algorithm函数动态切换调度算法，支持运行时配置。

2)抢占机制：通过preemptive标志控制优先级调度是否抢占（当前实现为非抢占式）。

(2) 同步与通信模块

1)信号量应用：消息队列的同步通过message_semaphore实现，确保线程安全的消息发送与接收。

互斥锁保护：process_mutex和message_mutex防止多线程环境下的竞态条件。

(3) 模拟运行模块

1)多线程架构：使用独立线程simulation_thread运行模拟循环，避免阻塞主线程。

2)时间驱动：通过current_time变量模拟系统时间，配合std::this_thread::sleep_for实现可视化延迟。

3.6 关键函数说明

- (1) add_process 添加新进程到系统，初始化 PCB 信息。
- (2) scheduler 调度器核心逻辑，根据算法选择下一个运行进程。
- (3) execute_time_unit 模拟进程执行一个时间单位，更新剩余时间和状态。
- (4) print_process_status 实时打印各进程状态及参数，用于调试和可视化。

4. 程序运行界面

(1) 初始界面：

```
===== 进程管理系统 =====
1. 添加进程
2. 设置调度算法
3. 设置时间片大小
4. 设置是否支持抢占
5. 运行模拟
6. 停止模拟
7. 查看当前状态
8. 发送消息
9. 接收消息
10. 信号量P操作(wait)
11. 信号量V操作(signal)
0. 退出
=====
请选择操作：|
```

(2) 添加进程：

进程1：

```
===== 进程管理系统 =====
1. 添加进程
2. 设置调度算法
3. 设置时间片大小
4. 设置是否支持抢占
5. 运行模拟
6. 停止模拟
7. 查看当前状态
8. 发送消息
9. 接收消息
10. 信号量P操作(wait)
11. 信号量V操作(signal)
0. 退出
=====
请选择操作：1
输入进程ID：01
输入到达时间：1
输入执行时间：100000
输入优先级(默认0)：0
进程已添加！
```

进程2：

```

===== 进程管理系统 =====
1. 添加进程
2. 设置调度算法
3. 设置时间片大小
4. 设置是否支持抢占
5. 运行模拟
6. 停止模拟
7. 查看当前状态
8. 发送消息
9. 接收消息
10. 信号量P操作(wait)
11. 信号量V操作(signal)
0. 退出
=====
请选择操作：1
输入进程ID：02
输入到达时间：1
输入执行时间：100000
输入优先级(默认0)：0
进程已添加！

```

(3) 选择调度算法：

```

===== 进程管理系统 =====
1. 添加进程
2. 设置调度算法
3. 设置时间片大小
4. 设置是否支持抢占
5. 运行模拟
6. 停止模拟
7. 查看当前状态
8. 发送消息
9. 接收消息
10. 信号量P操作(wait)
11. 信号量V操作(signal)
0. 退出
=====
请选择操作：2

选择调度算法：
1. 先来先服务(FCFS)
2. 短作业优先(SJF)
3. 时间片轮转(RR)
4. 优先级调度
请选择：1
调度算法已设置为：FCFS
调度算法已设置！

```

(4) 运行模拟：

```

=====
请选择操作：
PID      状态      到达时间      剩余时间      优先级
-----
1         新建        1              100000        0
2         新建        1              100000        0
-----
时间 1：进程 1 已到达并进入就绪队列
时间 1：进程 2 已到达并进入就绪队列
时间 1：进程 1 正在执行

===== 当前时间：1 =====
PID      状态      到达时间      剩余时间      优先级
-----
1         运行        1              99999         0
2         就绪        1              100000        0
-----
时间 2：进程 1 正在执行

===== 当前时间：2 =====
PID      状态      到达时间      剩余时间      优先级
-----
1         运行        1              99998         0
2         就绪        1              100000        0
-----
时间 3：进程 1 正在执行

```

(5) 查看当前状态:

```
===== 进程管理系统 =====
1. 添加进程
2. 设置调度算法
3. 设置时间片大小
4. 设置是否支持抢占
5. 运行模拟
6. 停止模拟
7. 查看当前状态
8. 发送消息
9. 接收消息
10. 信号量P操作(wait)
11. 信号量V操作(signal)
0. 退出
=====
请选择操作: 7

===== 当前时间: 6 =====
PID      状态      到达时间      剩余时间      优先级
-----
1         运行        1             99995         0
2         就绪        1             100000        0
-----
```

(6) 发送消息:

```
===== 进程管理系统 =====
1. 添加进程
2. 设置调度算法
3. 设置时间片大小
4. 设置是否支持抢占
5. 运行模拟
6. 停止模拟
7. 查看当前状态
8. 发送消息
9. 接收消息
10. 信号量P操作(wait)
11. 信号量V操作(signal)
0. 退出
=====
请选择操作: 8
输入要发送的消息: 123
进程 1 发送消息: 123
消息已发送!
```

(7) 接受消息:

```
===== 进程管理系统 =====
1. 添加进程
2. 设置调度算法
3. 设置时间片大小
4. 设置是否支持抢占
5. 运行模拟
6. 停止模拟
7. 查看当前状态
8. 发送消息
9. 接收消息
10. 信号量P操作(wait)
11. 信号量V操作(signal)
0. 退出
=====
请选择操作: 9
进程 1 收到来自进程 1 的消息: 123
收到消息: 123
```

5. 总结

5.1 实现成果

- (1) 成功模拟了多进程并发环境，支持四种调度算法动态切换，验证了不同算法对进程性能的影响（如 FCFS 的公平性、RR 的响应速度）。
- (2) 信号量机制有效解决了进程同步问题，消息队列实现了可靠的进程间通信。
- (3) 可视化输出清晰展示了进程状态流转、调度事件及统计结果，符合课设要求。

5.2 课程设计体会

通过本次设计，深入理解了操作系统进程管理的底层逻辑，掌握了从需求分析到代码实现的全流程开发能力。在解决多线程同步、算法逻辑冲突等问题时，提升了系统思维和调试技巧。同时，对进程调度算法的性能差异有了直观认识，为后续学习内存管理、文件系统等模块奠定了实践基础。